Stroma World v1.0 — Implementation Roadmap

> **Purpose**: This document is a sequenced set of AI-digestible prompts. Each section is a self-contained task you can hand to a coding AI (Claude, Copilot, Cursor, etc.). Complete them in order — later tasks depend on earlier ones.
> **Stack**: PhysiCell 1.14+ (C++17), Python 3.10+ (scripting/EA wrapper), VSCode with C/C++ and Python extensions.
> **Architecture overview**: PhysiCell runs the spatial multicellular simulation. A Python wrapper launches PhysiCell runs, parses outputs, evaluates fitness, and drives the evolutionary algorithm. A Boolean gene network inside each cell (C++ side) governs phenotype transitions. The EA (Python side) evolves intervention parameters.

Phase 0: Project Scaffolding & PhysiCell Setup

Prompt 0.1 — Directory Structure

Create the following project directory structure for a PhysiCell-based PDAC stroma simulation called "stroma_world":

```
stroma_world/
├── PhysiCell/                # PhysiCell source (git submodule or copy)
│   ├── core/
│   ├── modules/
│   └── ...
├── custom_modules/           # Our custom C++ code
│   ├── custom.h
│   ├── custom.cpp
│   ├── boolean_network.h     # Boolean gene network header
│   ├── boolean_network.cpp   # Boolean gene network implementation
│   ├── gene_definitions.h    # Gene enum, indices, and constants
```

```
│   ├── tumor_cell.h            # Tumor cell phenotype logic
│   ├── tumor_cell.cpp
│   ├── stromal_cell.h          # PSC/CAF phenotype logic
│   └── stromal_cell.cpp
├── config/
│   ├── PhysiCell_settings.xml    # Master PhysiCell config
│   ├── gene_params_default.json  # Default gene expression / mutation states
│   └── intervention_template.json # Template for EA-generated interventions
├── python/
│   ├── ea/
│   │   ├── __init__.py
│   │   ├── evolutionary_algorithm.py
│   │   ├── fitness.py
│   │   ├── population.py
│   │   └── operators.py          # Crossover, mutation, selection
│   ├── wrapper/
│   │   ├── __init__.py
│   │   ├── physicell_runner.py   # Launches PhysiCell, manages I/O
│   │   ├── config_generator.py   # Writes XML/JSON configs per EA individual
│   │   └── output_parser.py      # Parses PhysiCell MultiCellDS output
│   ├── analysis/
│   │   ├── __init__.py
│   │   └── visualize.py
│   └── run_ea.py               # Main entry point
├── output/                     # PhysiCell output (gitignored)
├── Makefile
├── requirements.txt            # Python deps
└── README.md
```

Create this structure with placeholder files. Each .h/.cpp file should have a header comment describing its role. Each .py file should have a module docstring. The Makefile should extend PhysiCell's default Makefile pattern to include our custom_modules. requirements.txt should list: numpy, scipy, deap (EA library), matplotlib, lxml, pandas, json5.

Prompt 0.2 — PhysiCell Configuration XML

Create a PhysiCell_settings.xml configuration file for a 2D PDAC tumor-stroma simulation with these specifications:

DOMAIN:
- 2D simulation (z disabled), 2000 x 2000 micron domain
- dx = dy = 20 microns (microenvironment voxel size)
- Mechanics voxel size: 30 microns
- Max simulation time: 14400 min (10 days)
- Output interval: 60 min (MultiCellDS), SVG every 60 min

MICROENVIRONMENT SUBSTRATES (5 diffusible fields):
1. oxygen — diffusion: 100000 μm^2/min, decay: 0.1 /min, initial/boundary: 38 mmHg
2. tgfb — diffusion: 1000 μm^2/min, decay: 0.01 /min, initial: 0, Dirichlet OFF
3. shh — diffusion: 1000 μm^2/min, decay: 0.01 /min, initial: 0, Dirichlet OFF
4. ecm_density — diffusion: 0.0 μm^2/min (non-diffusing field), decay: 0.0, initial: 0.1
5. drug — diffusion: 5000 μm^2/min, decay: 0.002 /min, initial: 0, Dirichlet OFF

CELL DEFINITIONS (2 types):

1. "tumor_cell" (ID 0):
   - Initial count: 50, placed in a cluster at center
   - Cycle: live model (code 5), default rate 0.00072 /min (~24h doubling)
   - Apoptosis rate: 5.316e-5 /min (default PhysiCell)
   - Volume: 2494 μm^3
   - Motility: enabled, speed 0.5 μm/min, persistence 10 min, 2D
   - Secretion: tgfb secretion rate 0.05, shh secretion rate 0.02
   - oxygen uptake: 10 /min
   - Mechanics: cell-cell adhesion 0.4, cell-cell repulsion 10.0
   - Custom data fields (doubles, all init 0.0 unless noted):
     KRAS (init 1.0), MYC, EGFR (init 1.0), TP53 (init 0.0 = lost),

BCL_XL, BAX, CDKN2A (init 0.0 = lost), SMAD4 (init 0.0 = lost),
RB1, ZEB1, CDH1, MMP2, TGFB1_expr, SHH_expr,
HIF1A, NRF2, ABCB1,
is_mesenchymal (0.0), stress_level (0.0),
drug_sensitivity (1.0), apoptosis_threshold (0.5)

2. "stromal_cell" (ID 1):
- Initial count: 200, scattered uniformly in domain
- Cycle: live model, rate 0.00036 /min (~48h doubling, quiescent baseline)
- Apoptosis rate: 1e-5 /min (long-lived)
- Volume: 2494 μm^3
- Motility: enabled, speed 0.1 μm/min, persistence 5 min, 2D
- Secretion: none by default (activated by Boolean network)
- oxygen uptake: 5 /min
- Mechanics: cell-cell adhesion 0.8, cell-cell repulsion 10.0
- Custom data fields:
  ACTA2 (init 0.0), GLI1 (init 0.0), HAS2 (init 0.0), COL1A1 (init 0.0),
  TGFB1_expr (init 0.0), MMP2 (init 0.0), HIF1A (init 0.0),
  is_activated (0.0), ecm_production_rate (0.0)

Add XML comments explaining each section. Use PhysiCell 1.14+ XML schema.

Phase 1: Gene Definitions & Boolean Network (C++)

Prompt 1.1 — Gene Enum and Constants

Create gene_definitions.h for the Stroma World simulation. This header defines all 21 genes as an enum, gene metadata, and constants.

Requirements:

1. An enum `GeneIndex` with entries for all 21 genes:
   KRAS=0, MYC=1, EGFR=2, TP53=3, BCL_XL=4, BAX=5, CDKN2A=6, SMAD4=7, RB1=8,
   ZEB1=9, CDH1=10, MMP2=11, ACTA2=12, TGFB1=13, SHH=14, GLI1=15, HAS2=16,
   COL1A1=17, HIF1A=18, NRF2=19, ABCB1=20, GENE_COUNT=21

2. A struct `GeneInfo` with fields:
   - name (std::string)
   - cell_type (enum: TUMOR, STROMA, BOTH)
   - functional_axis (enum: GROWTH, DEATH, BRAKING, INVASION, SECRETION, STRESS)
   - is_druggable (bool)
   - default_tumor_state (double, 0.0 or 1.0)
   - default_stroma_state (double, 0.0 or 1.0)

3. A static array `GENE_INFO[GENE_COUNT]` populated with the correct metadata for each gene based on this spec:
   - KRAS: TUMOR, GROWTH, druggable=false, tumor_default=1.0 (constitutively active)
   - MYC: TUMOR, GROWTH, druggable=false, tumor_default=0.5 (intermediate)
   - EGFR: TUMOR, GROWTH, druggable=true, tumor_default=1.0
   - TP53: TUMOR, DEATH, druggable=false, tumor_default=0.0 (lost in PDAC)
   - BCL_XL: TUMOR, DEATH, druggable=true (navitoclax), tumor_default=0.8
   - BAX: TUMOR, DEATH, druggable=false, tumor_default=0.3
   - CDKN2A: TUMOR, BRAKING, druggable=false, tumor_default=0.0 (lost)
   - SMAD4: TUMOR, BRAKING, druggable=false, tumor_default=0.0 (lost)
   - RB1: TUMOR, BRAKING, druggable=false, tumor_default=0.5
   - ZEB1: TUMOR, INVASION, druggable=false, tumor_default=0.0
   - CDH1: TUMOR, INVASION, druggable=false, tumor_default=1.0 (epithelial default)
   - MMP2: BOTH, INVASION, druggable=true, tumor_default=0.2, stroma_default=0.1
   - ACTA2: STROMA, INVASION, druggable=false, stroma_default=0.0 (quiescent)
   - TGFB1: BOTH, SECRETION, druggable=true, tumor_default=0.3, stroma_default=0.1
   - SHH: TUMOR, SECRETION, druggable=true (vismodegib), tumor_default=0.3
   - GLI1: STROMA, SECRETION, druggable=true, stroma_default=0.0
   - HAS2: STROMA, SECRETION, druggable=true (PEGPH20), stroma_default=0.0
   - COL1A1: STROMA, SECRETION, druggable=false, stroma_default=0.0

- HIF1A: BOTH, STRESS, druggable=false, tumor_default=0.0, stroma_default=0.0
- NRF2: TUMOR, STRESS, druggable=false, tumor_default=0.0
- ABCB1: TUMOR, STRESS, druggable=true, tumor_default=0.0

4. Constants for phenotype mapping thresholds:
- HYPOXIA_O2_THRESHOLD = 5.0 (mmHg, below this HIF1A activates)
- TGFB_ACTIVATION_THRESHOLD = 0.5 (tgfb substrate level to activate stroma)
- SHH_ACTIVATION_THRESHOLD = 0.5
- ECM_HIGH_THRESHOLD = 0.7 (ecm_density level considered "dense stroma")
- CONTACT_INHIBITION_THRESHOLD = 0.8 (fraction of max neighbors)
- EMT_ZEB1_THRESHOLD = 0.5 (above this, cell is mesenchymal)
- STRESS_NRF2_THRESHOLD = 0.5 (above this, drug resistance kicks in)

Use proper include guards, constexpr where possible, and C++17 features.

Prompt 1.2 — Boolean Network Engine

Create boolean_network.h and boolean_network.cpp that implement a continuous Boolean network for the Stroma World gene set.

DESIGN DECISIONS:
- Use CONTINUOUS values [0.0, 1.0] not discrete {0, 1}, so the EA can evolve graded interventions.
- Each gene has a "target state" computed by Boolean rules, then the actual state moves toward the target with a time constant (tau). This gives smooth dynamics.
- Update equation: gene[i] += dt * (target[i] - gene[i]) / tau[i]
- tau default = 6.0 min for signaling genes, 60.0 min for transcription factors, 360.0 min for structural proteins (COL1A1, HAS2).

CLASS: BooleanNetwork
- Member: double gene_states[GENE_COUNT] — current continuous state of each gene
- Member: double tau[GENE_COUNT] — time constants
- Member: bool is_mutant[GENE_COUNT] — if true, gene is locked (KRAS always ON, TP53 always OFF, etc.)

METHOD: void initialize(CellType cell_type, const GeneParams& params)
- Set default states from GENE_INFO based on cell type
- Lock mutant genes: for tumor cells, KRAS=1.0(locked), TP53=0.0(locked), CDKN2A=0.0(locked), SMAD4=0.0(locked)
- params is a struct loaded from config that can override defaults (for EA variation)

METHOD: void update(double dt, double oxygen, double tgfb_local, double shh_local, double drug_local)
- Computes target states from Boolean rules, then applies continuous update equation.
- This is called every PhysiCell mechanics timestep per cell.

BOOLEAN RULES FOR TUMOR CELLS:
(Each rule computes a target value in [0, 1]. Locked/mutant genes skip the rule.)

```
// Growth axis
target[MYC] = KRAS * (1.0 - 0.5 * (1.0 - EGFR))
// MYC driven by KRAS, modulated by EGFR. If EGFR is blocked, MYC drops ~50%.

// Death axis
target[BCL_XL] = 0.3 + 0.5 * KRAS + 0.2 * NRF2
// BCL-XL is basally expressed, boosted by KRAS and stress signaling
target[BAX] = 0.3 + 0.5 * TP53
// BAX is basally expressed, strongly induced by TP53 (which is OFF in PDAC)

// Braking axis
target[RB1] = CDKN2A * 0.7 + SMAD4 * 0.3
// RB1 requires upstream brakes. Both lost → RB1 functionally inactive.

// Invasion axis
target[ZEB1] = clamp(0.3 * tgfb_local + 0.3 * HIF1A + 0.2 * KRAS - 0.3 * CDH1_feedback, 0, 1)
// ZEB1 activated by TGF-β, hypoxia, KRAS; repressed by epithelial state
target[CDH1] = clamp(1.0 - ZEB1, 0, 1)
// CDH1 and ZEB1 are mutually repressive (bistable EMT switch)
```

```
  target[MMP2] = 0.2 * ZEB1 + 0.3 * HIF1A + 0.2 * KRAS
  // MMP2 secretion driven by invasion/stress signals

  // Secretion axis
  target[TGFB1] = 0.3 * KRAS + 0.2 * HIF1A + 0.1
  // Tumor secretes TGF-β constitutively, boosted by hypoxia
  target[SHH] = 0.3 * KRAS + 0.1

  // Stress axis
  target[HIF1A] = (oxygen < HYPOXIA_O2_THRESHOLD) ? clamp(1.0 - oxygen /
HYPOXIA_O2_THRESHOLD, 0, 1) : 0.0
  // HIF1A is a direct O2 sensor
  target[NRF2] = 0.2 * HIF1A + 0.3 * (drug_local > 0 ? 1.0 : 0.0) + 0.1 * KRAS
  // NRF2 activated by hypoxia, drug exposure, and KRAS
  target[ABCB1] = 0.5 * NRF2 + 0.3 * HIF1A
  // Drug efflux pump, downstream of stress signals

BOOLEAN RULES FOR STROMAL CELLS:
  // Activation axis (quiescent PSC → activated CAF)
  target[ACTA2] = clamp(0.6 * tgfb_local + 0.3 * GLI1, 0, 1)
  // α-SMA activation driven by TGF-β and Hedgehog signaling
  target[GLI1] = clamp(0.8 * shh_local, 0, 1)
  // GLI1 is the SHH signal transducer in stroma

  // ECM production (only when activated)
  target[HAS2] = 0.7 * ACTA2 + 0.2 * HIF1A
  target[COL1A1] = 0.8 * ACTA2
  target[MMP2] = 0.3 * ACTA2  // CAFs also produce some MMP2

  // Paracrine feedback
  target[TGFB1] = 0.4 * ACTA2  // Activated CAFs secrete more TGF-β (positive feedback)

  // Stress
```

target[HIF1A] = (oxygen < HYPOXIA_O2_THRESHOLD) ? clamp(1.0 - oxygen / HYPOXIA_O2_THRESHOLD, 0, 1) : 0.0

Include proper header guards, use PhysiCell's Cell* pointer for accessing custom data, and add detailed comments explaining each rule's biological rationale.

Prompt 1.3 — Intervention Interface

Add to boolean_network.h/.cpp an intervention system that lets the EA modulate gene activity:

STRUCT: Intervention
- gene_index (GeneIndex): which gene to target
- effect_type (enum): INHIBIT, ACTIVATE, KNOCKDOWN, OVEREXPRESS
- strength (double, 0.0-1.0): how strongly the intervention affects the gene
- drug_name (std::string): label for the intervention (for logging)

METHOD: void apply_interventions(const std::vector<Intervention>& interventions)
Called AFTER Boolean update. For each intervention:
- INHIBIT: gene_states[i] *= (1.0 - strength)
- ACTIVATE: gene_states[i] = gene_states[i] + strength * (1.0 - gene_states[i])
- KNOCKDOWN: is_mutant[i] = true; gene_states[i] = clamp(gene_states[i] - strength, 0, 1)
- OVEREXPRESS: is_mutant[i] = true; gene_states[i] = clamp(gene_states[i] + strength, 0, 1)

Note: INHIBIT/ACTIVATE are reversible (drug-like), KNOCKDOWN/OVEREXPRESS are permanent (gene therapy-like).
The strength parameter is what the EA evolves. The gene_index is selected from druggable genes.

METHOD: static std::vector<Intervention> load_from_json(const std::string& filepath)
Parse a JSON file like:
```
{
  "interventions": [
    {"gene": "EGFR", "effect": "INHIBIT", "strength": 0.8, "name": "erlotinib"},
    {"gene": "HAS2", "effect": "INHIBIT", "strength": 0.6, "name": "PEGPH20"},
```

```
    {"gene": "BCL_XL", "effect": "INHIBIT", "strength": 0.7, "name": "navitoclax"}
  ]
}
```

Use a simple JSON parser (nlohmann/json or manual parsing). Map gene name strings to GeneIndex enum values.

The intervention JSON file is what the Python EA generates for each individual in the population.

Phase 2: Phenotype Mapping (C++ — Connecting Genes to PhysiCell Behaviors)

Prompt 2.1 — Tumor Cell Phenotype Function

Create tumor_cell.h and tumor_cell.cpp that implement the phenotype update function for PDAC tumor cells.

This function is registered as a PhysiCell custom_cell_rule and called every mechanics timestep.

FUNCTION: void tumor_phenotype_update(Cell* pCell, Phenotype& phenotype, double dt)

It does three things in order:
1. Read microenvironment values at cell's position
2. Update the cell's Boolean network
3. Map gene states → PhysiCell phenotype parameters

STEP 1 — Read microenvironment:

```
double oxygen = pCell->nearest_density_vector()[oxygen_index];
double tgfb = pCell->nearest_density_vector()[tgfb_index];
double shh = pCell->nearest_density_vector()[shh_index];
double drug = pCell->nearest_density_vector()[drug_index];
// substrate indices set during setup from microenvironment names
```

STEP 2 — Update Boolean network:

```
BooleanNetwork* bn = get_boolean_network(pCell);  // retrieve from custom data or a cell-attached object
bn->update(dt, oxygen, tgfb, shh, drug);
bn->apply_interventions(current_interventions);  // global intervention list loaded at sim start
```

STEP 3 — Map genes to phenotype parameters:

```
// 3a. PROLIFERATION RATE
// Base rate * growth_drive, where growth_drive = f(KRAS, MYC, EGFR)
double growth_drive = bn->gene(KRAS) * (0.5 + 0.5 * bn->gene(MYC));
// Braking: if RB1 is active, reduce proliferation
double brake = 1.0 - 0.8 * bn->gene(RB1);
// Contact inhibition: check neighbor count vs threshold
double neighbor_fraction = (double)pCell->state.neighbors.size() / 8.0;  // approximate
if (bn->gene(CDKN2A) < 0.1 && bn->gene(SMAD4) < 0.1) {
    // Both brakes lost — ignore contact inhibition entirely
    brake = max(brake, 0.2);  // floor: always some growth
} else {
    if (neighbor_fraction > CONTACT_INHIBITION_THRESHOLD) {
        brake *= 0.1;  // severe growth arrest if brakes are functional
    }
}
phenotype.cycle.data.transition_rate(0, 0) = base_prolif_rate * growth_drive * brake;

// 3b. APOPTOSIS RATE
double bcl_xl = bn->gene(BCL_XL);
double bax = bn->gene(BAX);
double death_ratio = bax / (bcl_xl + 0.01);  // avoid division by zero
// TP53 lost → baseline apoptosis very low
double p53_factor = 0.2 + 0.8 * bn->gene(TP53);  // 0.2 baseline when TP53 is OFF
phenotype.death.rates[apoptosis_index] = base_apoptosis_rate * death_ratio * p53_factor;

// 3c. MOTILITY (EMT)
double zeb1 = bn->gene(ZEB1);
```

```
bool is_mesenchymal = (zeb1 > EMT_ZEB1_THRESHOLD);
pCell->custom_data["is_mesenchymal"] = is_mesenchymal ? 1.0 : 0.0;
if (is_mesenchymal) {
    phenotype.motility.migration_speed = 1.0;  // fast
    phenotype.mechanics.cell_cell_adhesion_strength = 0.1;  // low adhesion
} else {
    phenotype.motility.migration_speed = 0.25;
    phenotype.mechanics.cell_cell_adhesion_strength = 0.4;
}

// 3d. ECM DEGRADATION
double mmp2 = bn->gene(MMP2);
// Decrease ecm_density in the cell's voxel
int ecm_index = microenvironment.find_density_index("ecm_density");
double local_ecm = pCell->nearest_density_vector()[ecm_index];
pCell->nearest_density_vector()[ecm_index] = max(0.0, local_ecm - mmp2 * 0.01 * dt);

// 3e. SECRETION
double tgfb_secretion = bn->gene(TGFB1) * 0.05;
double shh_secretion = bn->gene(SHH) * 0.02;
phenotype.secretion.secretion_rates[tgfb_index] = tgfb_secretion;
phenotype.secretion.secretion_rates[shh_index] = shh_secretion;

// 3f. DRUG SENSITIVITY
double nrf2 = bn->gene(NRF2);
double abcb1 = bn->gene(ABCB1);
double drug_sensitivity = 1.0 - 0.4 * nrf2 - 0.4 * abcb1;
pCell->custom_data["drug_sensitivity"] = max(0.05, drug_sensitivity);

// 3g. DRUG-INDUCED DEATH (if drug is present)
if (drug > 0.01) {
    double effective_drug = drug * pCell->custom_data["drug_sensitivity"];
    // Increase apoptosis proportional to effective drug
```

```
    phenotype.death.rates[apoptosis_index] += effective_drug * 0.001;
}
```

Add detailed inline comments. Use PhysiCell API calls correctly (check PhysiCell docs for exact method signatures). Handle edge cases (division by zero, clamping to valid ranges).

Prompt 2.2 — Stromal Cell Phenotype Function

Create stromal_cell.h and stromal_cell.cpp implementing the phenotype update for PSC/CAF stromal cells.

FUNCTION: void stromal_phenotype_update(Cell* pCell, Phenotype& phenotype, double dt)

Same 3-step pattern as tumor cells, but different gene→phenotype mappings:

STEP 3 — Map genes to phenotype:

```
// 3a. ACTIVATION STATE
double acta2 = bn->gene(ACTA2);
bool is_activated = (acta2 > 0.5);
pCell->custom_data["is_activated"] = is_activated ? 1.0 : 0.0;

// 3b. PROLIFERATION
// Quiescent PSCs barely divide. Activated CAFs divide faster.
double gli1 = bn->gene(GLI1);
double activation_prolif = 0.1 + 0.9 * acta2;  // 10% baseline, up to 100% when fully activated
double hedgehog_boost = 1.0 + 0.5 * gli1;  // GLI1 boosts proliferation
phenotype.cycle.data.transition_rate(0, 0) = base_stroma_prolif_rate * activation_prolif * hedgehog_boost;

// 3c. ECM PRODUCTION
// Activated CAFs produce ECM (HAS2 → hyaluronan, COL1A1 → collagen)
double has2 = bn->gene(HAS2);
double col1a1 = bn->gene(COL1A1);
double ecm_production = (has2 * 0.5 + col1a1 * 0.5) * acta2;
```

```
pCell->custom_data["ecm_production_rate"] = ecm_production;

// Increase ecm_density in cell's voxel
int ecm_index = microenvironment.find_density_index("ecm_density");
double local_ecm = pCell->nearest_density_vector()[ecm_index];
pCell->nearest_density_vector()[ecm_index] = min(1.0, local_ecm + ecm_production * 0.005 * dt);

// 3d. MOTILITY
// Activated CAFs (myofibroblasts) are contractile and somewhat motile
if (is_activated) {
    phenotype.motility.migration_speed = 0.3;
} else {
    phenotype.motility.migration_speed = 0.05;  // quiescent, barely moves
}

// 3e. SECRETION (paracrine feedback)
double tgfb_secretion = bn->gene(TGFB1) * 0.03;  // activated CAFs secrete TGF-β
phenotype.secretion.secretion_rates[tgfb_index] = tgfb_secretion;

// 3f. MMP2 (CAFs also remodel ECM)
double mmp2 = bn->gene(MMP2);
// CAFs both produce AND degrade ECM — net effect depends on activation level
// But MMP2 from CAFs is much less than production, so stroma grows
pCell->nearest_density_vector()[ecm_index] = max(0.0,
    pCell->nearest_density_vector()[ecm_index] - mmp2 * 0.002 * dt);

// 3g. DRUG DIFFUSION MODULATION
// Dense ECM slows drug diffusion — this is handled at the microenvironment level,
// but we store the local ECM density for the fitness function
// (Actual diffusion modulation is in the custom microenvironment update, Prompt 2.3)
```

Same coding standards as tumor cell: inline comments, proper PhysiCell API, edge case handling.

Prompt 2.3 — ECM-Dependent Drug Diffusion

```
Implement a custom microenvironment update function that makes drug diffusion depend on local ECM density.

FUNCTION: void ecm_dependent_diffusion(double dt)
  Registered as a custom microenvironment function in PhysiCell setup.

LOGIC:
  For each microenvironment voxel:
    double ecm = density_vector[ecm_index];
    // Drug diffusion coefficient decreases with ECM density
    // D_drug(ecm) = D_drug_base * (1.0 - ecm_blocking_factor * ecm)
    // ecm_blocking_factor = 0.8, so at max ECM density (1.0), drug diffuses at 20% of normal
    double blocking = 1.0 - 0.8 * clamp(ecm, 0.0, 1.0);
    // Update the drug diffusion coefficient for this voxel
    // PhysiCell uses a uniform diffusion coefficient per substrate, so we approximate this
    // by modifying the drug decay rate locally (higher ECM → faster local drug decay)
    // This is the PhysiCell-compatible approximation.
    density_vector[drug_index] *= (1.0 - 0.8 * ecm * 0.001 * dt);

ALTERNATIVE APPROACH (if PhysiCell supports per-voxel diffusion in your version):
  Modify the diffusion coefficient directly using microenvironment.diffusion_coefficients[drug_index]
  on a per-voxel basis. Check PhysiCell version.

IMPORTANT: This function is what creates the "stroma barrier" — it's the mechanistic link between
CAF-produced ECM and drug delivery failure. Without it, the EA doesn't have a stroma barrier to
solve.

Also implement oxygen reduction in high-ECM regions:
  // Dense stroma → poor perfusion → lower O2
  // Increase O2 decay rate proportional to ECM density
  // This creates hypoxia in the tumor core, activating HIF1A
```

```
double o2_decay_boost = 1.0 + 2.0 * clamp(ecm, 0.0, 1.0);
// Apply as increased uptake or decay for oxygen in dense regions
```

Phase 3: Main Setup & Registration (C++)

Prompt 3.1 — Custom Module Setup

```
Create custom.h and custom.cpp that serve as the entry point for all custom modules in the PhysiCell simulation.

FUNCTION: void setup_microenvironment(void)
  - Find and store indices for all 5 substrates (oxygen, tgfb, shh, ecm_density, drug)
  - Store as global/static variables accessible by phenotype functions

FUNCTION: void create_cell_types(void)
  - Register "tumor_cell" with phenotype function tumor_phenotype_update
  - Register "stromal_cell" with phenotype function stromal_phenotype_update
  - Set up custom data fields for each cell type

FUNCTION: void setup_tissue(void)
  - Place 50 tumor cells in a circular cluster at domain center (radius ~50μm)
  - Place 200 stromal cells uniformly distributed across domain
  - Initialize each cell's BooleanNetwork based on cell type
  - Load interventions from JSON file (path from command-line args or config)

FUNCTION: void custom_function(Cell* pCell, Phenotype& phenotype, double dt)
  - Dispatcher: calls tumor_phenotype_update or stromal_phenotype_update based on cell type

Make sure to:
  - Include all necessary PhysiCell headers
  - Register custom_function via cell_defaults.functions.update_phenotype
  - Handle command-line argument for intervention JSON path
```

```
- Add timing/performance logging
- Store BooleanNetwork objects efficiently (one per cell, in custom data or a parallel data structure)
```

Prompt 3.2 — Makefile

```
Create a Makefile that builds the Stroma World PhysiCell project.

Requirements:
  - Extends PhysiCell's standard Makefile pattern
  - Compiles all files in custom_modules/
  - Links against PhysiCell core and modules
  - C++17 standard
  - Optimization flags: -O2 for release, -O0 -g for debug
  - Targets: all, clean, debug, release
  - Include path to nlohmann/json (or bundle a single-header JSON parser)
  - Output binary: stroma_world

The Makefile should also have a 'test' target that runs the simulation for 100 timesteps with default config
and checks that the output directory is populated.
```

Phase 4: Python EA Wrapper

Prompt 4.1 — PhysiCell Runner

```
Create python/wrapper/physicell_runner.py that manages launching and monitoring PhysiCell simulations.

CLASS: PhysiCellRunner
  __init__(self, binary_path, config_path, output_dir, timeout_seconds=3600)

  METHOD: run(self, intervention_json_path) -> SimulationResult
```

```
  1. Create a unique run directory under output_dir (using UUID or timestamp)
  2. Copy the base config XML to the run directory
  3. Write the intervention JSON to the run directory
  4. Launch the PhysiCell binary as a subprocess:
     ./stroma_world --config run_dir/config.xml --intervention run_dir/intervention.json --output run_dir/output/
  5. Monitor the subprocess (timeout, memory limits)
  6. On completion, return SimulationResult with:
     - run_dir path
     - exit_code
     - wall_time
     - output file paths

METHOD: run_batch(self, intervention_jsons: list, max_parallel: int = 4) -> list[SimulationResult]
  Run multiple simulations in parallel using multiprocessing.Pool or concurrent.futures.
  This is critical for EA performance — each generation needs ~50-100 simulations.

CLASS: SimulationResult
  - run_dir: Path
  - exit_code: int
  - wall_time: float
  - success: bool
  - output_files: dict  # {timestep: filepath}

Include proper error handling, logging, and cleanup of failed runs.
For HPC (SLURM): add a method run_batch_slurm() that submits jobs via sbatch and polls for completion.
```

Prompt 4.2 — Output Parser

```
Create python/wrapper/output_parser.py that reads PhysiCell MultiCellDS output files and extracts metrics needed
for the fitness function.

CLASS: OutputParser
  __init__(self, output_dir: Path)
```

```
METHOD: parse_final_state(self) -> SimulationMetrics
  Read the LAST timestep output (highest-numbered .xml file in output/).
  Extract:
  - total_tumor_cells: int (count cells of type 0)
  - total_stromal_cells: int (count cells of type 1)
  - live_tumor_cells: int (exclude dead/apoptotic)
  - activated_cafs: int (stromal cells with custom_data["is_activated"] > 0.5)
  - mesenchymal_tumor_cells: int (tumor cells with custom_data["is_mesenchymal"] > 0.5)
  - mean_ecm_density: float (average ecm_density across microenvironment)
  - max_ecm_density: float
  - mean_tumor_drug_sensitivity: float (average custom_data["drug_sensitivity"] of tumor cells)
  - tumor_extent: float (max distance between any two tumor cells — proxy for invasion)
  - stroma_barrier_score: float (mean ecm_density in annular region around tumor)
  - drug_penetration: float (mean drug concentration within tumor region)
  - hypoxic_fraction: float (fraction of tumor cells with HIF1A > 0.5)

METHOD: parse_timeseries(self) -> pd.DataFrame
  Read ALL timestep outputs and return a DataFrame with columns:
  [time, tumor_count, stroma_count, mean_ecm, drug_penetration, ...]
  This is for visualization and debugging.

HELPER: _read_physicell_xml(self, filepath) -> dict
  Use lxml to parse PhysiCell's MultiCellDS XML format.
  The cell data is typically in:
    <cellular_information> → <cell_populations> → <cell_population> → <custom>
  And microenvironment data in:
    <microenvironment> → <domain> → <mesh> and <variables>

HELPER: _compute_stroma_barrier_score(self, cell_positions, ecm_field)
  1. Find tumor centroid
  2. Define annular region (e.g., 100-300μm from centroid)
  3. Average ecm_density in that annulus
```

This quantifies how much stroma surrounds the tumor.

Use numpy for efficient computation. Handle missing data gracefully.

Prompt 4.3 — Fitness Function

Create python/ea/fitness.py that evaluates how well an intervention strategy works against the stroma barrier.

FUNCTION: compute_fitness(metrics: SimulationMetrics) -> float
The fitness function for Cycle 1 (stroma barrier disruption). Higher = better.

COMPONENTS (weighted sum):

1. TUMOR KILL (weight: 0.35)
   tumor_kill_score = 1.0 - (live_tumor_cells / initial_tumor_cells)
   Clamped to [0, 1]. Measures: did the intervention reduce tumor burden?

2. STROMA DISRUPTION (weight: 0.25)
   stroma_score = 1.0 - metrics.stroma_barrier_score
   Clamped to [0, 1]. Measures: was the ECM barrier degraded?

3. DRUG PENETRATION (weight: 0.20)
   penetration_score = metrics.drug_penetration / max_expected_drug
   Clamped to [0, 1]. Measures: can drug reach the tumor interior?

4. RESISTANCE PENALTY (weight: -0.10)
   resistance_penalty = 1.0 - metrics.mean_tumor_drug_sensitivity
   Measures: did the intervention inadvertently upregulate resistance?
   This is SUBTRACTED — interventions that trigger NRF2/ABCB1 are penalized.

5. INVASION PENALTY (weight: -0.10)
   invasion_penalty = metrics.tumor_extent / domain_size
   Measures: did stroma disruption cause tumor to spread?

```
    Critical: some stroma-disrupting strategies INCREASE invasion. This catches that.

  TOTAL:
  fitness = 0.35 * tumor_kill + 0.25 * stroma_disruption + 0.20 * penetration
            - 0.10 * resistance_penalty - 0.10 * invasion_penalty

  Return fitness clamped to [0, 1].

Also implement:

FUNCTION: compute_fitness_detailed(metrics: SimulationMetrics) -> dict
  Returns a dict with each component score for logging/analysis.

FUNCTION: validate_metrics(metrics: SimulationMetrics) -> bool
  Sanity checks: no negative cell counts, ECM in [0,1], etc.
  Returns False if simulation likely crashed or produced garbage.

Add docstrings explaining the biological rationale for each component and weight.
The weights are configurable via a FitnessConfig dataclass that can be loaded from JSON.
```

Prompt 4.4 — Evolutionary Algorithm

```
Create python/ea/evolutionary_algorithm.py that implements the main EA loop using the DEAP library.

CLASS: StromaWorldEA
  __init__(self, config: EAConfig)

  EAConfig dataclass:
    population_size: int = 50
    generations: int = 100
    mutation_rate: float = 0.2
    crossover_rate: float = 0.7
    tournament_size: int = 3
```

```
  max_interventions: int = 5  # max number of simultaneous drugs
  parallel_sims: int = 4
  druggable_genes: list = [EGFR, BCL_XL, MMP2, TGFB1, SHH, GLI1, HAS2, ABCB1]
  # Only genes marked druggable can be targeted

INDIVIDUAL REPRESENTATION:
Each individual is a list of Intervention dicts:
[
  {"gene": "EGFR", "effect": "INHIBIT", "strength": 0.73},
  {"gene": "HAS2", "effect": "INHIBIT", "strength": 0.45},
  ...
]
Variable length (1 to max_interventions).
The EA evolves: which genes to target, what effect type, and how strong.

METHOD: initialize_population(self) -> list
  Generate random individuals:
  - Random number of interventions (1 to max_interventions)
  - Random druggable gene for each
  - Random effect type (INHIBIT or ACTIVATE, weighted 80/20)
  - Random strength (uniform 0.1 to 1.0)

METHOD: evaluate(self, individual) -> float
  1. Convert individual to intervention JSON
  2. Run PhysiCell simulation via PhysiCellRunner
  3. Parse output via OutputParser
  4. Compute fitness via compute_fitness()
  5. Return fitness value

METHOD: mutate(self, individual) -> individual
  With probability mutation_rate, for each intervention:
  - 40%: mutate strength by ±0.1 (Gaussian)
  - 20%: change target gene (random druggable gene)
```

```
    - 10%: change effect type
    - 15%: add a new random intervention (if below max)
    - 15%: remove a random intervention (if above 1)

  METHOD: crossover(self, ind1, ind2) -> (ind1, ind2)
    Uniform crossover at the intervention level:
    - Pool all interventions from both parents
    - Each child randomly selects from the pool
    - Deduplicate (no two interventions on same gene)

  METHOD: run(self) -> EAResult
    Standard DEAP EA loop:
    1. Initialize population
    2. Evaluate all individuals (parallel)
    3. For each generation:
       a. Select parents (tournament selection)
       b. Apply crossover
       c. Apply mutation
       d. Evaluate offspring
       e. Replace population (elitism: keep top 5%)
       f. Log statistics (best/mean/worst fitness, best individual)
    4. Return EAResult with best individual, fitness history, full population history

  METHOD: save_checkpoint(self, generation, filepath)
  METHOD: load_checkpoint(self, filepath) -> generation

CLASS: EAResult
  best_individual: list[dict]
  best_fitness: float
  fitness_history: list[dict]  # per-generation stats
  population_history: list     # optional, full populations
  runtime_seconds: float
```

Use DEAP's toolbox registration pattern. Include detailed logging (generation stats to CSV).
Add a method to generate a summary report of the best intervention strategy found.

Prompt 4.5 — Main Entry Point

Create python/run_ea.py as the main entry point for running the evolutionary algorithm.

Usage:
  python run_ea.py --config ea_config.json --physicell-binary ./stroma_world --physicell-config config/PhysiCell_settings.xml --output-dir output/ea_run_001

Features:
  - Parse command-line arguments (argparse)
  - Load EA config from JSON
  - Initialize PhysiCellRunner, OutputParser, and StromaWorldEA
  - Run the EA
  - Save results:
    - best_intervention.json (the winning strategy)
    - fitness_history.csv
    - ea_result.json (full result object)
    - plots/ directory with fitness curves
  - Support resuming from checkpoint: --resume checkpoint.pkl
  - Print a human-readable summary of the best strategy found:
    "Best strategy (fitness=0.82):
     1. Inhibit EGFR at 73% strength (erlotinib-like)
     2. Inhibit HAS2 at 45% strength (PEGPH20-like)
     3. Inhibit BCL_XL at 91% strength (navitoclax-like)
     Interpretation: Disrupt stroma (HAS2) to allow drug penetration,
     block growth signaling (EGFR), and overcome death resistance (BCL-XL)."

  - For HPC: --slurm flag that uses run_batch_slurm() instead of local parallel

Include a --dry-run flag that generates 3 random individuals and prints their intervention JSONs without running simulations (for testing the pipeline).

Phase 5: Config Generator & Parameterization

Prompt 5.1 — Config Generator

```
Create python/wrapper/config_generator.py that generates per-simulation config files from EA individuals.

CLASS: ConfigGenerator
  __init__(self, base_config_xml: Path, base_gene_params: Path)

  METHOD: generate(self, individual: list[dict], run_dir: Path) -> tuple[Path, Path]
    1. Copy base PhysiCell XML config to run_dir
    2. Generate intervention.json from the individual's intervention list
    3. Optionally modify XML config if needed (e.g., drug substrate initial conditions)
    4. Return (config_xml_path, intervention_json_path)

  METHOD: generate_drug_protocol(self, individual: list[dict]) -> dict
    If the individual targets drug-related genes, set up the drug substrate:
    - For INHIBIT interventions, set drug boundary condition to strength * max_drug_concentration
    - For genes that don't need the drug substrate (e.g., genetic interventions), skip
    - Return a protocol dict that can be embedded in the XML config

Also create config/gene_params_default.json:
{
  "tumor_mutations": {
    "KRAS": {"locked": true, "state": 1.0},
    "TP53": {"locked": true, "state": 0.0},
    "CDKN2A": {"locked": true, "state": 0.0},
    "SMAD4": {"locked": true, "state": 0.0}
```

```
  },
  "tau_overrides": {},
  "initial_state_noise": 0.05,
  "description": "Default PDAC genotype: KRAS+, TP53-, CDKN2A-, SMAD4-"
}
```

And config/intervention_template.json:

```
{
  "interventions": [],
  "drug_delivery": {
    "start_time": 2880,
    "mode": "constant_boundary"
  },
  "notes": "Template - EA fills in interventions list"
}
```

Phase 6: Analysis & Visualization

Prompt 6.1 — Visualization Tools

```
Create python/analysis/visualize.py with plotting functions for analyzing EA results and simulation outputs.

FUNCTION: plot_fitness_history(fitness_csv: Path, output_path: Path)
  Line plot of best/mean/worst fitness per generation. Include standard deviation band.

FUNCTION: plot_intervention_frequency(population_history: list, output_path: Path)
  Bar chart showing how often each gene is targeted across the final population.
  This reveals which genes the EA "thinks" are most important.

FUNCTION: plot_gene_network_state(simulation_output: Path, timestep: int, output_path: Path)
  Heatmap of gene expression levels across all cells at a given timestep.
```

```
  Rows = genes, columns = cells, color = expression level [0, 1].

FUNCTION: plot_simulation_timeseries(timeseries_df: pd.DataFrame, output_path: Path)
  Multi-panel plot:
  - Panel 1: Tumor cell count over time
  - Panel 2: Mean ECM density over time
  - Panel 3: Drug penetration over time
  - Panel 4: Activated CAF fraction over time

FUNCTION: plot_stroma_barrier(simulation_output: Path, output_path: Path)
  2D spatial heatmap of ECM density with tumor cell positions overlaid.
  Shows the stroma barrier visually.

FUNCTION: compare_interventions(results: list[EAResult], labels: list[str], output_path: Path)
  Side-by-side comparison of different EA runs or intervention strategies.

Use matplotlib with a clean publication-style theme. All plots should be saved as PNG and PDF.
```

Phase 7: Integration Testing & Validation

Prompt 7.1 — Unit Tests

```
Create a test suite for the Stroma World project.

FILE: tests/test_boolean_network.py
  - Test that KRAS-locked tumor network produces expected MYC, BCL-XL levels
  - Test that TP53 loss reduces BAX expression
  - Test that high TGF-β activates stromal ACTA2
  - Test that HIF1A responds to low O2
  - Test that interventions (INHIBIT EGFR) reduce downstream MYC
  - Test that continuous update converges to target states
```

- Test edge cases: all genes OFF, all genes ON, simultaneous interventions

FILE: tests/test_fitness.py
- Test fitness function with known-good metrics (expect high fitness)
- Test fitness function with known-bad metrics (expect low fitness)
- Test that resistance penalty works (high NRF2 → lower fitness)
- Test that invasion penalty works (large tumor extent → lower fitness)
- Test boundary conditions (zero cells, max ECM, etc.)

FILE: tests/test_output_parser.py
- Create mock PhysiCell output files
- Test that parser extracts correct cell counts
- Test stroma barrier score computation
- Test timeseries parsing

FILE: tests/test_ea.py
- Test individual generation (valid interventions only)
- Test mutation (stays within bounds)
- Test crossover (no duplicate gene targets)
- Test that EA runs for 1 generation without crashing (using mock fitness)

Use pytest. Include fixtures for common test data.

Prompt 7.2 — Validation Simulations

Create python/validation/validate_biology.py that runs a set of known-outcome simulations to verify the model behaves biologically correctly.

VALIDATION SCENARIOS:

1. NO_INTERVENTION_BASELINE
   Run with no interventions. Expected:
   - Tumor grows

- Stroma activates (ACTA2 goes up over time)
- ECM density increases around tumor
- Drug (if added late) doesn't penetrate well

Criterion: stroma_barrier_score > 0.5 by day 10

2. KRAS_DEPENDENCE

Run with KRAS artificially set to OFF (mutation override).

Expected: tumor barely grows, low MYC, low proliferation.

Criterion: final tumor count < 1.5x initial

3. STROMA_DISRUPTION

Run with all stromal cells removed (0 initial stromal cells).

Expected: tumor grows uninhibited, no ECM barrier, drug penetrates fully.

Criterion: stroma_barrier_score < 0.1, drug_penetration > 0.8

4. TGF_BETA_BLOCKING

Run with TGFB1 INHIBIT at strength 1.0.

Expected: stroma activation is reduced, less ECM, but tumor still grows.

Criterion: activated_cafs < 30% of stroma cells

5. DUAL_THERAPY

Run with EGFR INHIBIT (0.8) + HAS2 INHIBIT (0.8).

Expected: some stroma disruption AND some growth inhibition.

Criterion: fitness > 0.5 (better than no intervention)

6. RESISTANCE_EMERGENCE

Run with strong drug exposure from start.

Expected: surviving tumor cells upregulate NRF2 and ABCB1.

Criterion: mean NRF2 of surviving cells > 0.5

Each scenario runs the simulation, parses output, checks criteria, and reports PASS/FAIL.

This is a regression test suite for the biological model.

Phase 8: HPC Deployment (LARCC/SLURM)

Prompt 8.1 — SLURM Integration

Create deployment scripts for running the EA on a SLURM HPC cluster.

FILE: hpc/submit_ea.sh

```
#!/bin/bash
#SBATCH --job-name=stroma_world_ea
#SBATCH --partition=gpu  (or appropriate partition)
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=32G
#SBATCH --time=24:00:00
#SBATCH --output=logs/ea_%j.out
#SBATCH --error=logs/ea_%j.err

module load gcc/12  # or appropriate compiler
module load python/3.10

cd $SLURM_SUBMIT_DIR
python python/run_ea.py \
  --config config/ea_config_hpc.json \
  --physicell-binary ./stroma_world \
  --physicell-config config/PhysiCell_settings.xml \
  --output-dir output/ea_$SLURM_JOB_ID \
  --parallel 16
```

FILE: hpc/submit_array.sh

For running multiple independent EA runs (different random seeds):

#SBATCH --array=1-10
Uses SLURM_ARRAY_TASK_ID as random seed.

FILE: python/wrapper/slurm_runner.py
Alternative to local PhysiCellRunner.run_batch() that:
1. Writes each simulation as a separate SLURM job script
2. Submits via subprocess("sbatch ...")
3. Polls sacct for completion
4. Collects results
This allows the EA to farm out simulations across multiple nodes.

Include environment setup, module loading, and path configuration for the University of Louisville LARCC system.

Appendix A: Gene-to-Code Quick Reference

#	Gene	Custom Data Key	Boolean Rules (Tumor)	PhysiCell Mapping
1	KRAS	KRAS	LOCKED=1.0	Base growth multiplier
2	MYC	MYC	= KRAS * f(EGFR)	Proliferation rate scaling
3	EGFR	EGFR	Input node (druggable)	Modulates MYC
4	TP53	TP53	LOCKED=0.0	Apoptosis threshold
5	BCL-XL	BCL_XL	= 0.3 + 0.5*KRAS* + *0.2*NRF2	Anti-apoptosis factor
6	BAX	BAX	= 0.3 + 0.5*TP53	Pro-apoptosis factor
7	CDKN2A	CDKN2A	LOCKED=0.0	Contact inhibition gate

#	Gene	Custom Data Key	Boolean Rules (Tumor)	PhysiCell Mapping
8	SMAD4	SMAD4	LOCKED=0.0	TGF-β brake (lost)
9	RB1	RB1	= CDKN2A*0.7* + *SMAD4*0.3	Cell cycle checkpoint
10	ZEB1	ZEB1	= f(TGFβ, HIF1A, KRAS, CDH1)	Motility switch
11	CDH1	CDH1	= 1 - ZEB1	Adhesion strength
12	MMP2	MMP2	= f(ZEB1, HIF1A, KRAS)	ECM degradation rate
13	ACTA2	ACTA2	[Stroma] = f(TGFβ, GLI1)	CAF activation switch
14	TGFB1	TGFB1_expr	= 0.3*KRAS* + *0.2*HIF1A + 0.1	TGF-β secretion rate
15	SHH	SHH_expr	= 0.3*KRAS + 0.1	SHH secretion rate
16	GLI1	GLI1	[Stroma] = 0.8*shh_local	Hedgehog signal transduction
17	HAS2	HAS2	[Stroma] = 0.7*ACTA2* + *0.2*HIF1A	HA production → ECM density
18	COL1A1	COL1A1	[Stroma] = 0.8*ACTA2	Collagen production → ECM density
19	HIF1A	HIF1A	= f(O_2 threshold)	Hypoxia response, VEGF proxy
20	NRF2	NRF2	= f(HIF1A, drug, KRAS)	Drug sensitivity modifier
21	ABCB1	ABCB1	= 0.5*NRF2* + *0.3*HIF1A	Drug efflux (intracellular drug decay)

Appendix B: Druggable Gene Targets for EA

Gene	Real Drug Analog	Effect Type	Strength Range	Notes
EGFR	Erlotinib	INHIBIT	0.3–1.0	Reduces MYC → slows growth
BCL-XL	Navitoclax	INHIBIT	0.3–1.0	Shifts BCL-XL/BAX ratio → more death
MMP2	Marimastat	INHIBIT	0.3–0.8	Reduces ECM degradation (careful: this can help OR hurt)
TGFB1	Galunisertib	INHIBIT	0.3–1.0	Reduces stroma activation
SHH	Vismodegib	INHIBIT	0.3–1.0	Blocks Hedgehog → reduces CAF proliferation
GLI1	GANT61	INHIBIT	0.3–1.0	Downstream Hedgehog block
HAS2	PEGPH20	INHIBIT	0.3–1.0	Reduces hyaluronan → improves drug diffusion
ABCB1	Verapamil	INHIBIT	0.3–0.8	Reduces drug efflux → more intracellular drug

Appendix C: Execution Order Summary

```
PHASE 0: Scaffold → Build directory structure, PhysiCell XML config
PHASE 1: Boolean network → Gene enum, network engine, intervention interface
PHASE 2: Phenotype mapping → Tumor/stroma update functions, ECM-drug coupling
PHASE 3: Integration → Main setup, cell registration, Makefile
PHASE 4: Python EA → Runner, parser, fitness, EA core, entry point
PHASE 5: Config generation → Per-run config files
PHASE 6: Visualization → Plots and analysis tools
```

PHASE 7: Testing → Unit tests + biological validation scenarios

PHASE 8: HPC → SLURM scripts for LARCC deployment

Each phase is self-contained. Compile and test after each phase before moving on. Phase 3 produces a runnable simulation (no EA). Phase 4+ adds the evolutionary optimization layer.